SUCTF2026 - SU_Artifact_Online - WriteUp

WriteUp

首先通过题目信息说提到的，nc进靶机后可以发现需要进行一个简单的pow验证，这里简单让ai写写脚本都能跑了，将爆破得到的S值提交即可进入正式的题目

```
=== Link Start ===
The artifact seems to require some verification before use.
sha256("bGpKByhzWzwNSY44" + S).hexdigest()[:6] == "000000"
S:
```

通过验证后，显示的ui如下，可以看到它展示了两个面，分别是正前面和右侧面，并且5x5的格子里各有一个符文，不确定的情况下，我们需要对这个玩意进行一些操作。可以留意到下方有两个模式选择，分别是翻转和激活

```
[40.0s]  A mysterious artifact. Try to find its secret.

        [Front]              [Right]
     ᛃ  ᚹ  ᛋ  ᛞ  ᛟ  |   ᚹ  ᛇ  ᛃ  ᚠ  ᛏ
     ᛗ  ᛣ  ᛒ  ᛞ  ᚢ  |   ᛃ  ᚠ  ᚺ  ᛏ  ᚲ
     ᛣ  ᚺ  ᛋ  ᚺ  ᚲ  |   ᚲ  ᚠ  ᛃ  ᛊ  ᚹ
     ᛝ  ᚱ  ᚠ  ᚠ  ᚢ  |   ᛉ  ᛣ  ᛗ  ᛗ  ᛉ
     ᚱ  ᚨ  ᛋ  ᛇ  ᛈ  |   ᛈ  ᛝ  ᛈ  ᚠ  ᛜ

  1 -- Try to twist it
  2 -- Try to activate it
  q -- Leave

>
```

选择翻转，可以进入新的界面，可以看到上面提示了通过以下命令来进行一些翻转操作，并可以通过加上`符号来执行逆操作

```
R1-R5 / C1-C5 / F1-F5
```

其中R对应的是正前面的行，1-5则分别对应第一行到第五行，正向操作是进行顺时针翻转，也就是right面的行会朝着front面移动，逆操作则相反

C对应的是正前面的列，1-5则分别对应第一列到第五列，正向操作是进行顺时针翻转，也就是front面的列会向下翻转移动，逆操作则相反

F对应的是则是右侧面的列，1-5则分别对应第一列到第五列，正向操作是进行顺时针翻转，也就是right面的列会向下翻转移动，逆操作则相反

```
[30.0s]  --- Twist ---
R1-R5 / C1-C5 / F1-F5 (add ' to reverse) | q=back

        [Front]              [Right]
     ⍀  ⍗  ⍊  ⍈  ⍜  |   ⍗  ⍐  ⍀  ⍓  ⍑
     ⍌  ⍋  ⍇  ⍈  ⍊  |   ⍀  ⍓  ⍍  ⍑  ⍲
     ⍋  ⍍  ⍊  ⍍  ⍲  |   ⍲  ⍓  ⍀  ⍎  ⍗
     ⍘  ⍅  ⍭  ⍭  ⍊  |   ⍦  ⍋  ⍌  ⍌  ⍦
     ⍅  ⍑  ⍊  ⍐  ⍏  |   ⍏  ⍘  ⍏  ⍓  ⍜

move>
```

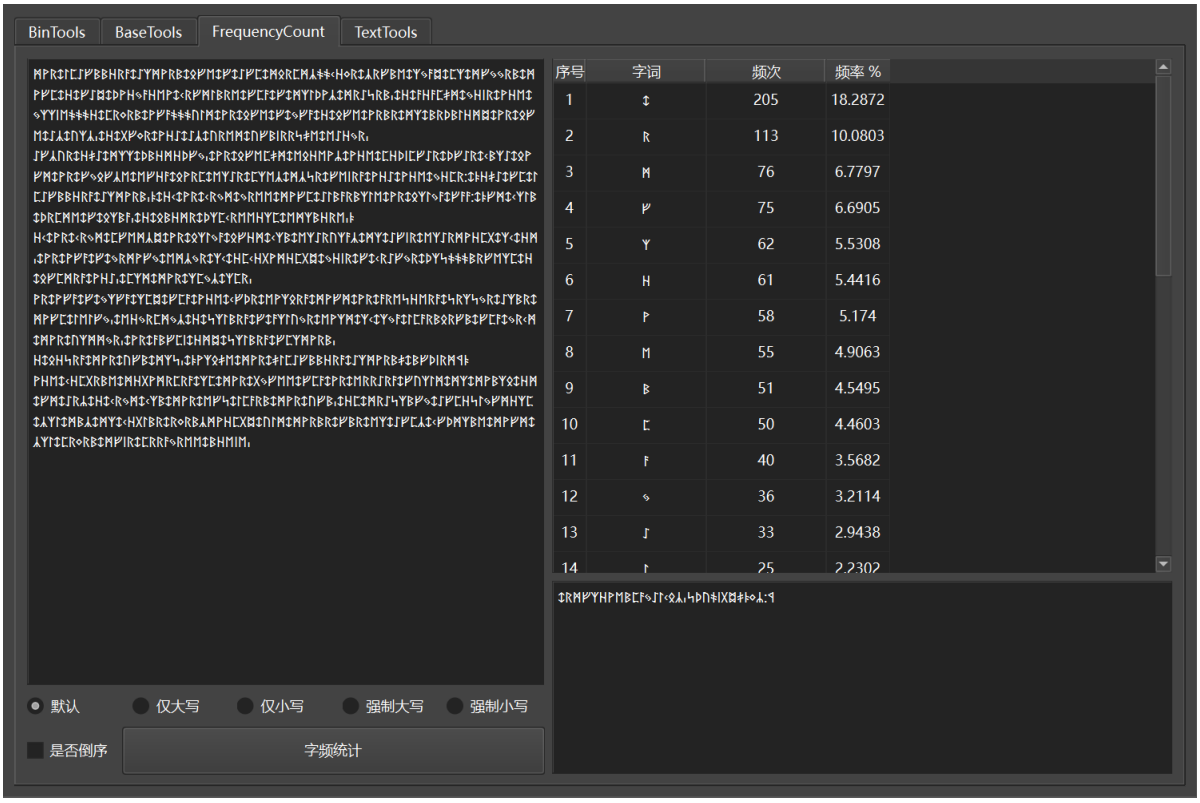
而选择激活，则会将魔方面锁定在正前面，以正前面作为激活面，进行横纵字符选择，从横向移动开始确认字符，确认后移动方向会转换成纵向，随后循环往复这个过程，在这个过程中需要选择合适的字符来构造对应的linux命令

```
[21.1s]  === Activate ===
Dir: Horizontal <- ->  Enter=pick  x=activate  q=quit

     0   1   2   3   4
 0   ⍀   ⍗   ⍊   ⍈   ⍜
 1   ⍌   ⍋   ⍇   ⍈   ⍊
 2   ⍋   ⍍   ⍊   ⍍   ⍲
 3   ⍘   ⍅   ⍭   ⍭   ⍊
 4   ⍅   ⍑   ⍊   ⍐   ⍏

  Cursor: ⍀
  Spell: (empty)
```

在确定魔方的一些操作后，就是要确定符文代表的字符了，题目中提供了一个附件，里面是一长串经过符文映射后的密文，其实通过简单的字符统计就可以发现里面只有大概31种，并且有明显的分行，大抵能推断出是由英文字符+一些符号构造出来的文段经由映射变成的密文

```
ᴍᴘʀ↨ᴛᴄᴊᴘᴢʙʜʀꜰ↨ᴊʏᴍᴘʀʙ↨ᴘᴍ↨ᴘ↨ᴊᴘᴄ↨ᴍᴘʀᴄᴍᴧ↑↑<ʜ◇ʀ↨ᴧʀᴘʙᴍ↨ʏ◇ꜰ⊠↨ᴄʏ↨ᴍᴘ◇◇ʀʙ↨ᴍᴘᴘᴄ↨ʜ↨ᴘᴊ⊠↨ᴅᴘʜ◇ꜰʜᴍᴘ↨<ʀᴘᴍᴛʙʀ
ᴍ↨ᴘᴄꜰ↨ᴘ↨ᴍʏᴛᴅᴘᴧ↨ᴍʀᴊᴧʀʙ,↨ʜ↨ꜰʜꜰᴄ↑ᴍ↨◇ʜᴛʀ↨ᴘʜᴍ↨◇ʏʏᴛᴍ↑↑↑↑ʜ↨ᴄʀ◇ʀʙ↨ᴘᴘᴘꜰ↑↑↑↑ᴛᴍ↨ᴘʀ↨ᴘᴍ↨ᴘ↨◇ᴘꜰ↨ʜ↨ᴘᴍ↨ᴘʀʙʀ↨ᴍʏ↨ʙ
ʀᴅʙᴛʜᴍ⊠↨ᴘʀ↨ᴘᴍ↨ᴊ↨ᴛᴧᴛ↨ʜ↨ᴘᴘ◇ʀ↨ᴘʜᴊ↨ᴊ↨ᴛᴧᴛ↨ʀᴍᴍ↨ᴛᴧᴘʙᴛʀʀʜ↑↑ᴍ↨ᴍᴊʜ◇ʀ,
ᴊᴘᴧᴛʀ↨ʜ↑↑↨ᴍʏʏ↨ᴅʙʜᴍʜᴅᴘ◇,↨ᴘʀ↨ᴘᴍᴄ↑ᴍ↨ᴍ↨ʜᴍᴘᴧ↨ᴘʜᴍ↨ᴄʜᴅᴄᴘᴊʀ↨ᴅᴘᴊʀ↨<ʙʏᴊ↨ᴘᴘ
ᴘᴍ↨ᴘʀ↨ᴘ◇ᴘᴧᴍ↨ᴍᴘʜꜰ↨ᴘʀᴄ↨ᴍʏᴊʀ↨ᴄʏᴍᴧ↨ᴍᴧʜʀ↨ᴘᴍᴘʀ↨ᴘʜᴊ↨ᴘʜᴍ↨◇ʜᴄʀ:↨ʜ↑↑↨ᴘᴄ↨ᴛ
ᴄᴊᴘʙʙʜʀꜰ↨ᴊʏᴍᴘʀʙ,↑↑ʜ<↨ᴘʀ↨<ʀ◇ᴍ↨◇ʀᴍᴍ↨ᴍᴘᴘᴄ↨ᴛᴛʙꜰʀʙʏᴛᴍ↨ᴘʀ↨ᴘʏ◇ꜰ↨ᴘꜰꜰ:↨ᴘᴍ↨<ʏʙ
↨ᴅʀᴄᴍᴍ↨ᴘ↨ᴘʏʙꜰ,↨ʜ↨ᴘʜᴍʀ↨ᴅʏᴄ<ʀᴍᴍʜʏᴄ↨ᴍᴍʏʙʜʀᴍ,↑
ʜ<↨ᴘʀ↨<ʀ◇ᴍ↨ᴄᴘᴍᴍᴧ⊠↨ᴘʀ↨ᴘʏ◇ꜰ↨ᴘʜᴍ↨<ʏʙ↨ᴍʏᴊʀᴍʏꜰᴧ↨ᴍʏ↨ᴊᴘᴊʀ↨ᴍʏᴊʀᴍᴘʜᴄᴘ↨ʏ<↨ʜᴍ
,↨ᴘʀ↨ᴘᴘꜰ↨ᴘ↨◇ʀᴍᴘᴘ◇↨ᴍᴍᴧ◇ʀ↨ʏ<↨ʜᴄ<ʜᴘᴍʜᴄᴘ⊠↨◇ʜᴛʀ↨ᴘ↨<ʀᴊᴘ◇ʀ↨ᴅʏ↑↑↑↑ʙʀᴘᴍʏᴄ↨ʜ
↨ᴘᴄᴍʀꜰ↨ᴘʜᴊ,↨ᴄʏᴍ↨ᴍᴘʀ↨ʏᴄ◇ᴧ↨ʏᴄʀ,
ᴘʀ↨ᴘᴘꜰ↨ᴘ↨◇ʏᴘꜰ↨ʏᴄ⊠↨ᴘᴄꜰ↨ᴘʜᴍ↨<ᴘʀ↨ᴍᴘʏ↨ʀꜰ↨ᴍᴘᴘ↨ᴍ↨ᴘʀ↨ꜰʀᴍ↑↑ʜᴍʀꜰ↑↑ʀʏ↑↑◇ʀ↨ᴊʏʙʀ↨
ᴍᴘᴘᴄ↨ᴍᴍᴘ◇,↨ᴍʜ◇ʀᴄᴍ◇ᴧ↨ʜ↑↑ʏʙʀꜰ↨ᴘꜰʏᴛᴛ◇ʀ↨ᴍᴘʏᴍʏ<↨ʏ◇ꜰ↨ᴄꜰʀʙ◇ʀᴘʙ↨ᴘᴄꜰ↨◇ʀ<ᴍ
↨ᴍᴘʀ↨ᴛʏᴍᴍ◇ʀ,↨ᴘʀ↨ꜰʙᴘᴄ↨ʜᴍ⊠↨↑ʏʙʀꜰ↨ᴘᴄʏᴍᴘʀʙ,
ʜ↨ᴅʜ↑↑ʀꜰ↨ᴍᴘʀ↨ᴛᴘᴘʙ↨ᴍʏ↑↑,↨ᴘʏ↨ᴛᴍᴘʀ↨ᴛᴛᴄᴊᴘʙʙʜʀꜰ↨ᴊʏᴍᴘʀʙ↨ᴛʙᴘᴅʀᴍ↑↑ʜ
ᴘʜᴍ↨<ʜᴄᴘʀʙᴍ↨ᴍʜᴘᴍᴘʀᴄʀꜰ↨ᴘᴄ↨ᴍᴘʀ↨ᴘ◇ᴍᴍ↨ᴘᴄꜰ↨ᴘʀ↨ᴍʀᴊʀꜰ↨ᴘᴛᴛᴍ↨ᴍʏ↨ᴍᴘʙʏ↨ʜᴍ
↨ᴘᴍ↨ᴊʀᴧ↨ʜ↨<ʀ◇ᴍ↨<ʏʙ↨ᴍᴘʀ↨ᴍᴘᴘ↑↑↨ᴛᴄꜰʀʙ↨ᴍᴘʀ↨ᴛᴘʙ,↨ʜᴄ↨ᴍʀᴊ↑↑ʏʙᴘ◇↨ᴊᴘᴄʜ↑↑◇ᴘᴍʜʏᴄ
↨ʏᴛ↨ᴍʙᴧ↨ᴍʏ↨<ʜᴛᴛʙʀ↨ᴛ◇ʀʙᴧᴍᴘʜᴄᴘ⊠↨ᴛᴛᴛ↨ᴍᴘʀʙʀ↨ᴘʙʀ↨ᴍʏ↨ᴊᴘᴄᴧ↨<ᴘᴅᴍʏʙᴍ↨ᴍᴘᴘᴍ↨
ᴧʏᴛ↨ᴄʀ◇ʀʙ↨ᴍᴘᴘᴍ↨ᴛᴄʀʀꜰ◇ʀᴍᴍ↨ʙʜᴍᴛᴍ,
```


对其进行词频统计，可以发现 ↨ 出现最多，大概率就是空格



将 ↨ 替换为空格后，用脚本替换一下字母和符号

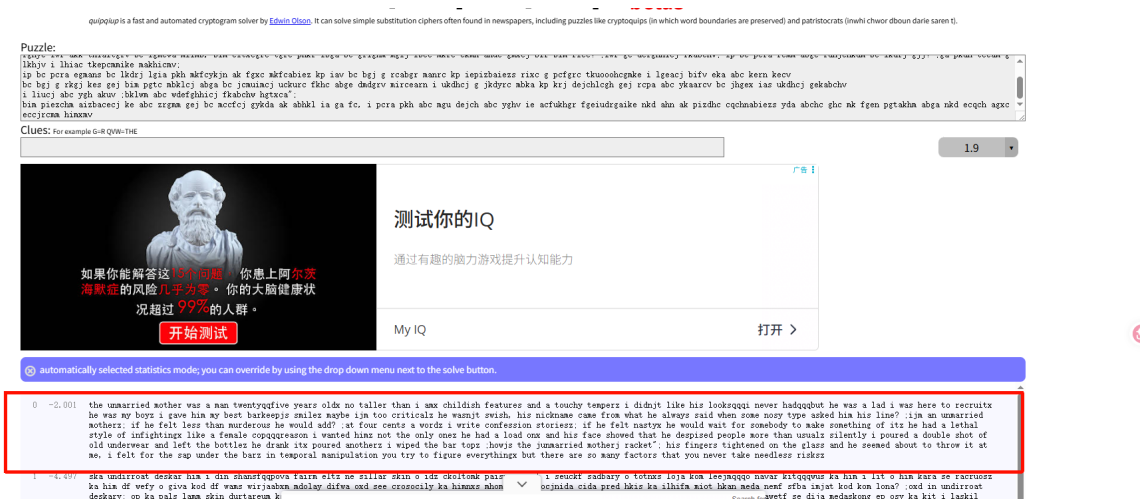
```python
symbols = "abcdefghijklmnopqrstuvwxyz,?;\"\'-!"  # 这里只需要写入26个字符加任意符号即可

with open("text", "r", encoding="utf-8") as f:
    text = f.read()

unique_chars = []
for ch in text:
    if ch in [" ", ".",'\n']:
        continue
    if ch not in unique_chars:
        unique_chars.append(ch)

if len(unique_chars) > len(symbols):
    raise ValueError("字符种类超过可映射数量")
```

```
mapping = {ch: symbols[i] for i, ch in enumerate(unique_chars)}

result = []
for ch in text:
    if ch == "\n":
        result.append("\n")
    elif ch == " ":
        result.append(" ")
    elif ch == ".":
        result.append(".")
    else:
        result.append(mapping[ch])

output = "".join(result)

print(output)
```

然后得到了

```
abc defghhicj fkabch lgm g fge alceanoopiqc ncghm krjs ek agrrch abge i gfs
tbirjimb pcgadhcm gej g akdtbn acfuchv i jijewa rixc bim rkkxmoooi ecqch
bgjoooyda bc lgm g rgj i lgm bchc ak hcthdias bc lgm fn yknv i zgqc bif fn ycma
yghxccuwm mfircv
fgnyc iwf akk thiaitgrv bc lgmewa mlimb, bim eitxegfc tgfc phkf lbga bc grlgnm
mgij lbce mkfc ekmn anuc gmxcj bif bim riec? ;iwf ge defghhicj fkabchv; ip bc
pcra rcmm abge fdhjchkdm bc lkdrj gjj? ;ga pkdh tceam g lkhjv i lhiac tkepcmmike
makhicmv;
ip bc pcra egmans bc lkdrj lgia pkh mkfcykjn ak fgxc mkfcabiez kp iav bc bgj g
rcabgr manrc kp iepizbaiezs rixc g pcfgrc tkuooohcgmke i lgeacj bifv eka abc kern
kecv
bc bgj g rkgj kes gej bim pgtc mbklcj abga bc jcmuimcj uckurc fkhc abge dmdgrv
mircearn i ukdhcj g jkdyrc mbka kp krj dejchlcgh gej rcpa abc ykaarcv bc jhgex
ias ukdhcj gekabchv
i liucj abc ygh akuv ;bklwm abc wdefghhicj fkabchw hgtxca";
bim piezchm aizbacecj ke abc zrgmm gej bc mccfcj gykda ak abhkl ia ga fc, i pcra
pkh abc mgu dejch abc yghv ie acfukhgr fgeiudrgaike nkd ahn ak pizdhc cqchnabiezs
yda abchc ghc mk fgen pgtakhm abga nkd ecqch agxc eccjrcmm himxmv
```

然后拿到quipqiup里进行词频分析，则可以还原出大概得明文是什么，通过社工或者问ai就能锁定明文是来自Robert A. Heinlein的短篇小说《All You Zombies》



找到原文对应的部分，然后将他跟密文一一对应，即可还原映射表，简单拿脚本还原一下

```
from collections import OrderedDict

def build_mapping(plain, cipher):
    plain = plain.replace("\n", "")
    cipher = cipher.replace("\n", "")

    if len(plain) != len(cipher):
        raise ValueError(f"长度不一致: plain={len(plain)} cipher={len(cipher)}")

    mapping = OrderedDict()

    for p, c in zip(plain, cipher):
        p = p.lower()

        if p not in mapping:
            mapping[p] = c
        else:
            if mapping[p] != c:
                raise ValueError(f"映射冲突: '{p}' -> '{mapping[p]}' 与 '{c}'")

    return mapping


def print_mapping(mapping):
    print("{")
    for k, v in mapping.items():
        if k == "'":
            k = "\\'"
        print(f"    '{k}':'{v}',")
    print("}")


if __name__ == "__main__":
    plaintext = """The Unmarried Mother was a man twenty--five years old, no
taller than I am, childish features and a touchy temper. I didn't like his looks-
--I never had---but he was a lad I was here to recruit, he was my boy. I gave him
my best barkeep's smile.
Maybe I'm too critical. He wasn't swish; his nickname came from what he always
said when some nosy type asked him his line: "I'm an unmarried mother." If he
felt less than murderous he would add: "at four cents a word. I write confession
stories."
If he felt nasty, he would wait for somebody to make something of it. He had a
lethal style of infighting, like a female cop---reason I wanted him. Not the only
one.
He had a load on, and his face showed that he despised people more than usual.
Silently I poured a double shot of Old Underwear and left the bottle. He drank
it, poured another.
I wiped the bar top. "How's the 'Unmarried Mother' racket?"

His fingers tightened on the glass and he seemed about to throw it at me; I felt
for the sap under the bar. In temporal manipulation you try to figure everything,
but there are so many factors that you never take needless risks."""
```

```
    ciphertext =
"""ᛗᚠᚱᛏᚩᚳᛇᛈᛒᛒᚻᚱᚠᛏᛇᛉᛗᚠᚱᛒᛏᛟᛈᛗᛏᛈᛏᛇᛈᛏᛗᛟᚱᚳᛗᚣᛝᛝᚲᚻᛜᚱᛏᚣᚱᛈᛗᛏᛉᛜᚠᛗᛏᚳᛉᛏᛗᛈᛜᛜᚱᛒᛏᛗᚠᛈᛏᛏᛗᛟᚠᚻᛗᚠᛏᚲᚱᛈ
ᛗᛚᛒᚱᛗᛏᛈᚳᚠᛏᛈᛏᛗᛉᛚᚹᛈᛏᛗᚱᛇᛋᚱᛒᛚᛏᚻᛏᚠᚻᚠᚳᚠᛗᛏᛜᚻᛚᚱᛏᚹᚻᛏᛗᛏᛈᚻᛏᛈᛏᛗᛏᛈᛏᛗᛏᛈᛏᛗ
```

分别是`cd ..;nl flag`和`cd ..;cat flag`，为了保证一定有可执行解，实际上我在源码里预设了`cd ..;nl flag`的路径，不过`cd ..;cat flag`也是可以构造的（存在一定偶然性，用nl的准确率会高点）

最终exp如下，这里我直接用了全符文映射表，但是只靠附件给的量也是能构建出来的

```
#!/usr/bin/env python3
import hashlib
import itertools
import re
import socket
import ssl
import string
import sys
import time
from collections import deque

rmap = {
    'a':'ᴯ','b':'ᴨ','c':'ᴩ','d':'ᴦ','e':'ᴫ','f':'ᴣ','g':'ᴸ','h':'ᴾ',
    'i':'ᴴ','j':'ᴶ','k':'ᴵ','l':'ᴳ','m':'ᴲ','n':'ᴱ','o':'ᴹ','p':'ᴻ',
    'q':'ᴼ','r':'ᴽ','s':'ᴭ','t':'ᴭ','u':'ᴬ','v':'ᴰ','w':'ᴷ','x':'ᴭ',
    'y':'ᴺ','z':'ᴹ',',':'ᴮ',';':'ᴺ','.':'ᴬ',' ':'ᴯ','?':'ᴻ',
    '{':'ᴾ','}':'ᴾ','-':'ᴶ','"':'ᴷ','!':'ᴵ','\'':'ᴶ',
}
rrev = {v: k for k, v in rmap.items()}

def decode(text):
    return "".join(rrev.get(c, c) for c in text)

def solve_pow(prefix):
    charset = string.ascii_letters + string.digits
    print(f'[*] Solving PoW: sha256("{prefix}" + S)[:6] == "000000"')
    t0, cnt = time.time(), 0
    for length in range(1, 20):
        for combo in itertools.product(charset, repeat=length):
            s = "".join(combo)
            if hashlib.sha256((prefix + s).encode()).hexdigest()[:6] == "000000":
                print(f'[+] PoW solved: S="{s}" ({cnt} attempts, {time.time()-
t0:.2f}s)')
                return s
            cnt += 1
            if cnt % 5_000_000 == 0:
                e = time.time() - t0
                print(f"  [pow] {cnt/1e6:.1f}M attempts, {cnt/e/1e6:.2f}M/s,
{e:.1f}s")

cmd = "cd ..;nl flag"
sz = 5
fnames = ["F", "R", "B", "L", "U", "D"]
all_moves = [f"{p}{i}{s}" for p in "RCF" for i in range(1, 6) for s in ("", "'")]
allpos = [(f, r, c) for f in fnames for r in range(sz) for c in range(sz)]
orbsz = {(0,0):6, (0,1):24, (0,2):24, (1,1):24, (1,2):48, (2,2):24}
arrows = {"up": b"\x1b[A", "down": b"\x1b[B", "right": b"\x1b[C", "left":
b"\x1b[D"}
ansi_re = re.compile(r"\x1b\[[0-9;]*[A-Za-z]")
```

```python
grid_re = re.compile(
    r'([\u16A0-\u16EB](?:\s+[\u16A0-\u16EB]){4})'
    r'\s+\|\s+'
    r'([\u16A0-\u16EB](?:\s+[\u16A0-\u16EB]){4})'
)
mapall_seq = (
    [f"R{i}" for i in range(1, 6)] + [f"R{i}" for i in range(1, 6)] +
    [f"R{i}'" for i in range(1, 6)] + [f"R{i}'" for i in range(1, 6)] +
    [f"C{i}'" for i in range(1, 6)] + [f"C{i}" for i in range(1, 6)] +
    [f"C{i}" for i in range(1, 6)] + [f"C{i}'" for i in range(1, 6)]
)
mapall_idx = {4: ("B", False), 9: ("L", False), 24: ("U", True), 34: ("D", True)}

def orb(c, r):
    a, b = abs(c - 2), abs(r - 2)
    return (min(a, b), max(a, b))

def inv(m):
    return m[:-1] if m.endswith("'") else m + "'"

class Conn:
    def __init__(self, host, port, use_ssl=False):
        raw = socket.create_connection((host, port), timeout=30)
        if use_ssl:
            ctx = ssl.create_default_context()
            ctx.check_hostname = False
            ctx.verify_mode = ssl.CERT_NONE
            self.s = ctx.wrap_socket(raw, server_hostname=host)
        else:
            self.s = raw
        self.buf = b""

    def recvuntil(self, delim, timeout=15):
        dl = time.time() + timeout
        while delim not in self.buf:
            rem = dl - time.time()
            if rem <= 0: break
            self.s.settimeout(max(rem, 0.1))
            try:
                d = self.s.recv(4096)
                if not d: break
                self.buf += d
            except socket.timeout:
                break
        i = self.buf.find(delim)
        if i >= 0:
            r = self.buf[:i + len(delim)]
            self.buf = self.buf[i + len(delim):]
            return r
        r = self.buf
        self.buf = b""
        return r

    def send(self, data):
        self.s.sendall(data + b"\n")
```

```
    def close(self):
        self.s.close()

def parse_grid(raw):
    clean = ansi_re.sub("", raw)
    fr, rr = [], []
    for line in clean.splitlines():
        m = grid_re.search(line)
        if m:
            fr.append([decode(c) for c in m.group(1).split()])
            rr.append([decode(c) for c in m.group(2).split()])
    return (fr[-sz:], rr[-sz:]) if len(fr) >= sz else (None, None)

def batch_send(r, mvs):
    if mvs:
        r.s.sendall(b"\n".join(m.encode() for m in mvs) + b"\n")

def batch_recv(r, n):
    return [r.recvuntil(b"move>", timeout=10).decode(errors="replace") for _ in
range(n)]

def read_faces(resps, front, right):
    st = {"F": front, "R": right}
    for idx, (name, use_f) in mapall_idx.items():
        f, r = parse_grid(resps[idx])
        g = f if use_f else r
        if g is None: return None
        st[name] = g
    return st

def cyc(t):
    return (t[1], t[2], t[3], t[0])

def deduce_perm(s0, s1, s2, s3):
    perm, used = {}, set()
    keys = {}
    for p in allpos:
        fn, r, c = p
        keys[p] = (s0[fn][r][c], s1[fn][r][c], s2[fn][r][c], s3[fn][r][c])
    for p in allpos:
        k = keys[p]
        if k[0] == k[1] == k[2] == k[3]:
            perm[p] = p
            used.add(p)
    di = {}
    for p in allpos:
        if p in used: continue
        di.setdefault(cyc(keys[p]), []).append(p)
    amb = []
    for p in allpos:
        if p in perm: continue
        k = keys[p]
        if k[0] == k[1] == k[2] == k[3]: continue
        cands = [q for q in di.get(k, []) if q not in used]
        if len(cands) == 1:
            perm[p] = cands[0]
```

```
            used.add(cands[0])
        elif cands:
            amb.append((p, cands))
    for p, cands in amb:
        if p in perm: continue
        fn, r, c = p
        o = orb(c, r)
        oc = [q for q in cands if q not in used and orb(q[2], q[1]) == o]
        if oc:
            perm[p] = oc[0]
            used.add(oc[0])
    for p in allpos:
        if p not in perm:
            perm[p] = p
    return perm

def probe_perms(conn, s0):
    perms = {}
    seq = list(mapall_seq)
    for px in "RCF":
        for i in range(1, 6):
            base = f"{px}{i}"
            ib = inv(base)
            batch = [base] + seq + [base] + seq + [base] + seq + [base] + seq +
[ib] * 4
            batch_send(conn, batch)
            resps = batch_recv(conn, 168)
            states, ok = [], True
            for j in range(3):
                off = j * 41
                f, r = parse_grid(resps[off])
                if f is None: ok = False; break
                st = read_faces(resps[off+1:off+41], f, r)
                if st is None: ok = False; break
                states.append(st)
            if not ok or len(states) < 3: continue
            pm = deduce_perm(s0, states[0], states[1], states[2])
            perms[base] = pm
            perms[base + "'"] = {v: k for k, v in pm.items()}
    return perms

def build_comms(perms):
    comms = {}
    for m1 in all_moves:
        m1i = inv(m1)
        p1, p1i = perms[m1], perms[m1i]
        for m2 in all_moves:
            if m2 == m1 or m2 == m1i: continue
            m2i = inv(m2)
            p2, p2i = perms[m2], perms[m2i]
            res, nontrivial = {}, False
            for p in allpos:
                d = p2i[p1i[p2[p1[p]]]]
                res[p] = d
                if d != p: nontrivial = True
            if nontrivial:
```

```
                comms[(m1, m2)] = (res, [m1, m2, m1i, m2i])
    return comms

def safe_ops(frozen, perms, comms=None):
    ops = [(perms[t], [t]) for t in all_moves if all(perms[t][p] == p for p in
frozen)]
    if comms:
        ops += [(pm, mv) for (_, _), (pm, mv) in comms.items()
                if all(pm[p] == p for p in frozen)]
    return ops

def bfs(start, goal, ops):
    if start == goal: return []
    vis = {start: None}
    q = deque([start])
    while q:
        cur = q.popleft()
        for pm, ms in ops:
            nxt = pm[cur]
            if nxt not in vis:
                vis[nxt] = (cur, ms)
                if nxt == goal:
                    path, n = [], nxt
                    while vis[n] is not None:
                        prev, m = vis[n]
                        path = list(m) + path
                        n = prev
                    return path
                q.append(nxt)
    return None

def copy_state(st):
    return {n: [row[:] for row in st[n]] for n in fnames}

def apply_moves(st, tokens, perms):
    for tok in tokens:
        nf = {n: [[None] * sz for _ in range(sz)] for n in fnames}
        for (sf, sr, sc), (df, dr, dc) in perms[tok].items():
            nf[df][dr][dc] = st[sf][sr][sc]
        for n in fnames:
            st[n] = nf[n]

def solve(state, path, perms, comms):
    orderings = [
        sorted(path, key=lambda x: orbsz[orb(x[1][0], x[1][1])]),
        sorted(path, key=lambda x: -orbsz[orb(x[1][0], x[1][1])]),
        list(path), list(reversed(path)),
        sorted(path, key=lambda x: (x[1][0], orbsz[orb(x[1][0], x[1][1])])),
        sorted(path, key=lambda x: (orbsz[orb(x[1][0], x[1][1])], x[1][0])),
        sorted(path, key=lambda x: x[1][1]),
    ]
    seen = set()
    for order in orderings:
        k = tuple(order)
        if k in seen: continue
        seen.add(k)
```

```
        sim = copy_state(state)
        frozen, seq, ok = set(), [], True
        for ch, (tc, tr) in order:
            tp = ("F", tr, tc)
            to = orb(tc, tr)
            cands = [(f, r, c) for f in fnames for r in range(sz) for c in
range(sz)
                     if sim[f][r][c] == ch and orb(c, r) == to]
            if not cands: ok = False; break
            cands.sort(key=lambda p: 0 if p[0] == 'F' else 1)
            found = False
            for uc in [False, True]:
                ops = safe_ops(frozen, perms, comms if uc else None)
                for cd in cands:
                    res = bfs(cd, tp, ops)
                    if res is not None:
                        apply_moves(sim, res, perms)
                        seq.extend(res)
                        frozen.add(tp)
                        found = True
                        break
                if found: break
            if not found: ok = False; break
        if not ok: continue
        if all(sim["F"][r][c] == ch for ch, (c, r) in path):
            return seq
    return None

def find_paths(state):
    needed = {}
    for ch in cmd:
        needed[ch] = needed.get(ch, 0) + 1
    avail = {}
    for ch in needed:
        av = {}
        for fn in fnames:
            for r in range(sz):
                for c in range(sz):
                    if state[fn][r][c] == ch:
                        o = orb(c, r)
                        if o != (0, 0):
                            av[o] = av.get(o, 0) + 1
        avail[ch] = av
    paths, used = [], set()
    def dfs(idx, cx, cy, hz, cur):
        if idx == len(cmd):
            paths.append(cur[:])
            return
        ch = cmd[idx]
        if ch not in avail: return
        for c in range(sz):
            for r in range(sz):
                if hz and r != cy: continue
                if not hz and c != cx: continue
                if (c, r) in used: continue
                o = orb(c, r)
```

```
                if o == (0, 0): continue
                if avail[ch].get(o, 0) == 0: continue
                used.add((c, r))
                cur.append((ch, (c, r)))
                dfs(idx + 1, c, r, not hz, cur)
                cur.pop()
                used.discard((c, r))
    dfs(0, 0, 0, True, [])
    paths.sort(key=lambda p: (
        -len(set(c for _, (c, _) in p)),
        -len(set(r for _, (_, r) in p)),
        sum(orbsz[orb(c, r)] for _, (c, r) in p)
    ))
    return paths

def drain(conn, timeout=0.6):
    chunks = [conn.buf]
    conn.buf = b""
    dl = time.time() + timeout
    while True:
        rem = dl - time.time()
        if rem <= 0: break
        conn.s.settimeout(max(rem, 0.05))
        try:
            d = conn.s.recv(4096)
            if not d: break
            chunks.append(d)
            dl = time.time() + 0.3
        except socket.timeout:
            break
    conn.buf = b""
    return b"".join(chunks)

def key_wait(conn, key):
    conn.s.sendall(key)
    conn.recvuntil(b"Cursor:", timeout=3)
    conn.recvuntil(b"\n", timeout=1)
    conn.recvuntil(b"\n", timeout=1)

def navigate(conn, coords):
    acts, cx, cy, hz = [], 0, 0, True
    for tx, ty in coords:
        if hz:
            rd, ld = (tx - cx) % sz, (cx - tx) % sz
            acts.append(("right", rd) if rd <= ld else ("left", ld))
            cx = tx
        else:
            dd, ud = (ty - cy) % sz, (cy - ty) % sz
            acts.append(("down", dd) if dd <= ud else ("up", ud))
            cy = ty
        hz = not hz
    print(f"  [nav] {len(coords)} chars, {sum(c for _, c in acts)} key presses")
    conn.send(b"2")
    got = conn.recvuntil(b"Cursor:", timeout=8)
    if b"Cursor:" not in got:
        return "(error)", "(error)"
```

```
    conn.recvuntil(b"\n", timeout=1)
    conn.recvuntil(b"\n", timeout=1)
    for si, (d, n) in enumerate(acts):
        for _ in range(n):
            key_wait(conn, arrows[d])
        conn.s.sendall(b"\r")
        conn.recvuntil(b"Cursor:", timeout=3)
        conn.recvuntil(b"\n", timeout=1)
        sp = conn.recvuntil(b"\n", timeout=1)
        print(f"  [nav] step {si+1}/{len(acts)}: {ansi_re.sub('',
sp.decode(errors='replace')).strip()}")
    conn.s.sendall(b"x")
    raw = drain(conn, timeout=4.0)
    text = ansi_re.sub("", raw.decode(errors="replace"))
    runes, plain, out = [], [], False
    for line in text.splitlines():
        s = line.strip()
        if not s: continue
        if "activating" in s: out = True; continue
        if out:
            if any(k in s for k in ["Enter", "continue", "Press", "hums"]): break
            runes.append(s)
            plain.append(decode(s))
    conn.s.sendall(b"\r")
    time.sleep(0.3)
    return "\n".join(runes) or "(no output)", "\n".join(plain) or "(no output)"

def main():
    if len(sys.argv) < 3:
        print(f"Usage: {sys.argv[0]} <host> <port> [--ssl]")
        sys.exit(1)
    host, port = sys.argv[1], int(sys.argv[2])
    use_ssl = "--ssl" in sys.argv
    t0 = time.time()
    conn = Conn(host, port, use_ssl)

    raw = conn.recvuntil(b"S: ", timeout=15).decode(errors="replace")
    m = re.search(r'sha256\("([^"]+)"\s*\+\s*S\)', raw)
    if not m:
        print("[-] bad pow"); conn.close(); return
    ans = solve_pow(m.group(1))
    if not ans:
        print("[-] pow failed"); conn.close(); return
    conn.send(ans.encode())
    data = conn.recvuntil(b"> ", timeout=15).decode(errors="replace")
    if "OK" not in data:
        print("[-] pow rejected"); conn.close(); return

    front, right = parse_grid(data)
    if front is None:
        print("[-] parse failed"); conn.close(); return
    conn.send(b"1")
    conn.recvuntil(b"move>", timeout=10)
    batch_send(conn, mapall_seq)
    resps = batch_recv(conn, 40)
    s0 = read_faces(resps, front, right)
```

```
    if s0 is None:
        print("[-] faces failed"); conn.close(); return

    perms = probe_perms(conn, s0)
    if len(perms) < 30:
        print(f"[-] only {len(perms)} perms"); conn.close(); return

    comms = build_comms(perms)
    paths = find_paths(s0)
    if not paths:
        print("[-] no paths"); conn.close(); return

    sol, used_path, tried = None, None, 0
    for p in paths[:500]:
        if time.time() - t0 > 30: break
        tried += 1
        ms = solve(s0, p, perms, comms)
        if ms is not None:
            sol, used_path = ms, p
            break
    if sol is None:
        print(f"[-] no solution ({tried} tried)"); conn.close(); return
    print(f"[+] Solution: {len(sol)} moves, tried {tried} paths ({time.time()-
t0:.1f}s)")

    batch_send(conn, sol)
    batch_recv(conn, len(sol))
    conn.send(b"q")
    conn.recvuntil(b"> ", timeout=10)

    coords = [(c, r) for _, (c, r) in used_path]
    rout, pout = navigate(conn, coords)
    elapsed = time.time() - t0

    print(f"\n{'='*50}")
    print(f"Moves: {len(sol)} | Time: {elapsed:.1f}s")
    print(f"\nRune:\n{rout}")
    print(f"\nPlain:\n{pout}")
    print(f"{'='*50}")
    try:
        conn.send(b"q")
    except Exception:
        pass
    conn.close()

if __name__ == "__main__":
    main()
```